软 件 设 计 文 档

跃动 MoveMate
数据库设计说明书

面向健身入门人群的个性化运动计划智能体（Web 应用）

文档编号	：MoveMate-DBD-002
项目名称	：跃动 MoveMate · 个性化运动计划智能体
团队名称	：跃动 MoveMate
文档版本	：V1.0
编写日期	：2026 年 11 月
密　　级	：内部

修订记录

版本	日期	修订说明	修订人	审核人
V0.1	2026-10-26	初稿：概念模型与实体梳理	(待填)	(待填)
V0.9	2026-11-05	补充关系模型、ORM 映射、物理设计	(待填)	(待填)
V1.0	2026-11-08	评审定稿	(待填)	(待填)

目录

- 1 引言
- 2 数据库总体设计
- 3 概念结构设计（ER 图）
- 4 逻辑结构设计（关系数据模型）
- 5 对象关系映射（ORM）
- 6 物理结构设计
- 7 领域知识与交互记录存储设计
- 8 数据安全与备份
- 附录 A 建表 SQL（节选）

1 引言

1.1 编写目的

本文档是《跃动 MoveMate》软件系统的数据库设计说明书，描述系统的概念结构（ER 图）、逻辑结构（关系数据模型）、对象关系映射（ORM）以及物理存储方案。文档遵循《数据库设计说明书》国标规范文本结构，面向课程评审教师、助教与项目组开发人员，作为数据库建模与实现的依据。

1.2 术语与缩写定义

缩写 / 术语	全称与说明
ER	Entity-Relationship，实体-关系模型，用于概念结构设计。
ORM	Object-Relational Mapping，对象关系映射，将实体类映射到库表。
PK / FK	Primary Key 主键 / Foreign Key 外键。
RAG	Retrieval-Augmented Generation，检索增强生成，依赖向量检索。
JSONB	PostgreSQL 二进制 JSON 类型，适合存储结构化处方。
FITT-VP	运动处方六要素（频率/强度/时间/类型/总量/进阶）。
3NF	Third Normal Form，第三范式。

1.3 参考资料

- 《跃动 MoveMate 系统设计说明书》（本文档的配套设计文档）；
- 《跃动 MoveMate 选题报告与需求分析》（第一次团队作业）；
- PostgreSQL 14 官方文档（JSONB、pgvector 扩展）。

2 数据库总体设计

2.1 设计目标与原则

- **一致性**：以领域模型为中心，业务实体与库表一一对应，保证各模块数据口径统一；
- **完整性**：主键、外键、唯一约束、检查约束齐备，保证数据质量；
- **规范化**：关系模式达到 3NF，消除冗余；对高频变动的处方结构使用 JSONB 适度反规范化，兼顾查询性能；
- **可扩展**：模型、知识库、工具均可替换，schema 预留扩展字段；
- **安全**：健康数据加密、最小化收集、权限分级。

2.2 数据库选型

主数据库选用 **PostgreSQL 14+**，并启用 **pgvector** 扩展，实现「业务数据 + 向量检索」一体化：

存储	技术	用途
业务数据库	PostgreSQL	用户、档案、处方、打卡、会话、知识库、日志
向量库	pgvector（同库扩展）	知识切片向量，供 RAG 相似度检索
缓存	Redis	会话缓存、热点数据、限流
对象存储	OSS	课件、动作图片等大文件（库中仅存 URL）

2.3 存储架构

分层数据流：业务数据（PostgreSQL）承载系统状态；领域知识经切片后以「文档元数据 + 切片向量」存于 PostgreSQL/pgvector，供 Agent 检索；会话与工具调用记录落库，支撑审计与可溯源；大文件走对象存储，库中仅存引用地址。

3 概念结构设计（ER 图）

3.1 ER 图

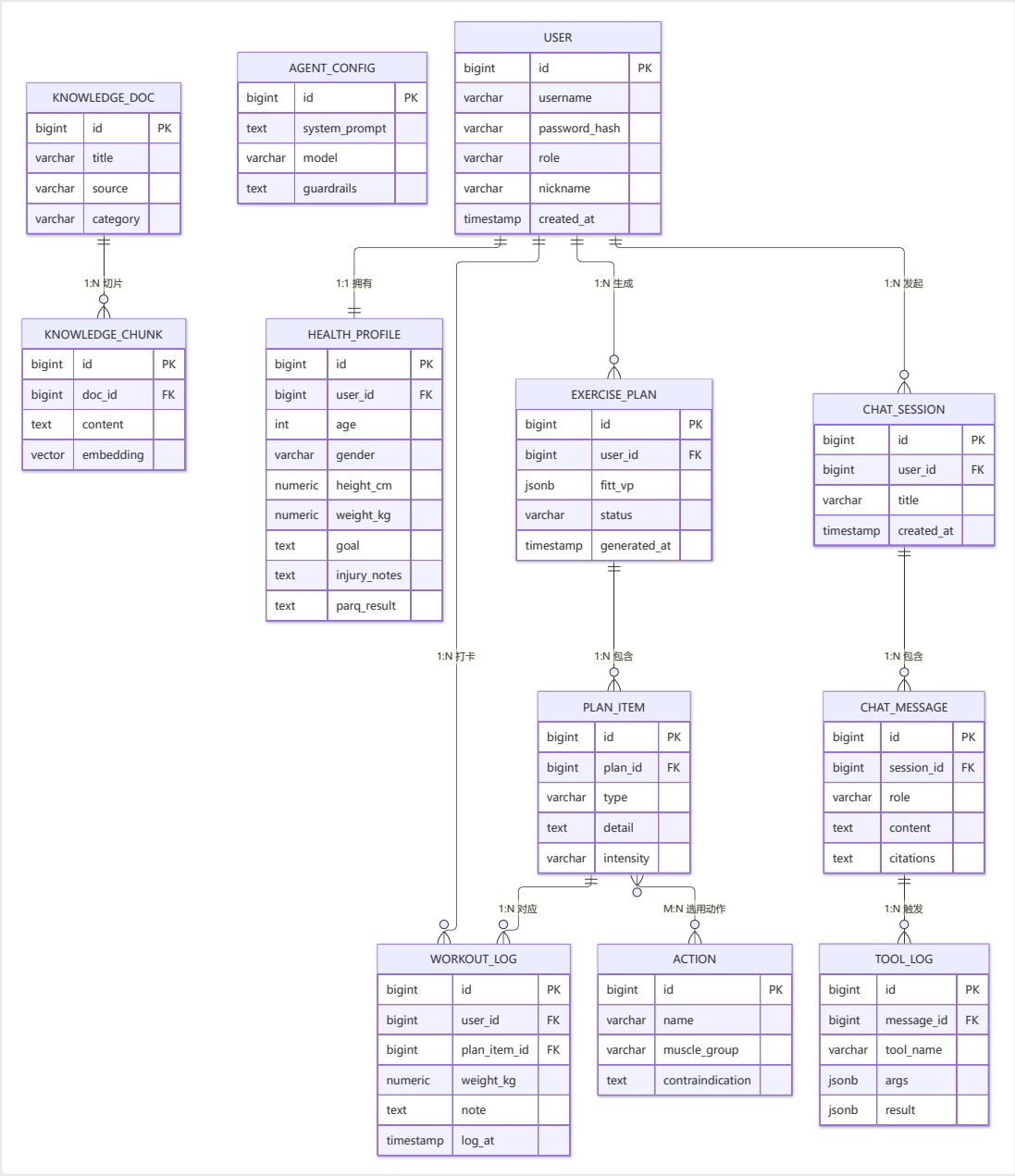


图 3-1 跃动 MoveMate 实体关系图（ER 图）

3.2 实体与关系说明

实体	含义	关系（基数）
USER	用户（健身用户/管理员）	1:1 拥有 HEALTH_PROFILE
HEALTH_PROFILE	健康档案与评估结果	1:1 属于 USER
EXERCISE_PLAN	运动处方	USER 1:N 生成；1:N 包含 PLAN_ITEM
PLAN_ITEM	处方训练条目	M:N 选用 ACTION
WORKOUT_LOG	打卡与身体数据记录	USER 1:N、PLAN_ITEM 1:N
CHAT_SESSION / CHAT_MESSAGE	智能体会话与消息	USER 1:N、SESSION 1:N
KNOWLEDGE_DOC / KNOWLEDGE_CHUNK	领域知识文档与向量切片	1:N 切片
ACTION	动作库字典	M:N 关联 PLAN_ITEM
TOOL_LOG	工具调用日志	CHAT_MESSAGE 1:N 触发
AGENT_CONFIG	Agent 配置（单例）	独立配置表

4 逻辑结构设计（关系数据模型）

将 ER 图转化为关系模式。命名约定：库表名用**小写下划线**，主键统一为 `bigint id`，外键统一为 `表名_id`。下面对每张表列出字段、类型与约束（PK=主键，FK=外键，UQ=唯一，NN=非空）。

4.1 user（用户表）

字段	类型	约束	说明
id	bigint	PK	用户主键
username	varchar(50)	UQ, NN	登录名
password_hash	varchar(255)	NN	密码哈希（加密）
role	varchar(20)	NN	角色：user / admin
nickname	varchar(50)		昵称
created_at	timestamp	NN	创建时间

4.2 health_profile（健康档案表）

字段	类型	约束	说明
id	bigint	PK	主键
user_id	bigint	FK→user, UQ	所属用户（1:1）
age	int		年龄
gender	varchar(10)		性别
height_cm	numeric(5,1)		身高（cm）
weight_kg	numeric(5,1)		体重（kg）
goal	text		训练目标
injury_notes	text		旧伤/禁忌描述
parq_result	text		PAR-Q 筛查结果（JSON）

4.3 exercise_plan（运动处方表）

字段	类型	约束	说明
id	bigint	PK	主键
user_id	bigint	FK→user, NN	所属用户
fitt_vp	jsonb	NN	FITT-VP 结构化处方
status	varchar(20)	NN	状态：draft/active/archived
generated_at	timestamp	NN	生成时间

4.4 plan_item（处方条目表）

字段	类型	约束	说明
id	bigint	PK	主键
plan_id	bigint	FK→exercise_plan, NN	所属处方
type	varchar(20)	NN	类型：有氧/抗阻/柔韧等
detail	text		训练细节（动作、组次）
intensity	varchar(50)		强度（心率区间/RPE）

4.5 workout_log（打卡记录表）

字段	类型	约束	说明
id	bigint	PK	主键
user_id	bigint	FK→user, NN	用户
plan_item_id	bigint	FK→plan_item	对应训练条目（可空）
weight_kg	numeric(5,1)		当日体重
note	text		训练感受备注
log_at	timestamp	NN	打卡时间

4.6 chat_session / chat_message（会话与消息表）

字段	类型	约束	说明
id	bigint	PK	会话主键（chat_session）
user_id	bigint	FK→user, NN	发起用户
title	varchar(200)		会话标题
id	bigint	PK	消息主键（chat_message）
session_id	bigint	FK→chat_session, NN	所属会话
role	varchar(20)	NN	user / assistant / tool
content	text	NN	消息内容
citations	text		引用出处（知识库条目）

4.7 knowledge_doc / knowledge_chunk（知识库表）

字段	类型	约束	说明
id	bigint	PK	文档主键（knowledge_doc）
title	varchar(255)	NN	文档标题
source	varchar(255)		来源（指南/教材）
category	varchar(50)		分类（处方学/强度量化等）
id	bigint	PK	切片主键（knowledge_chunk）
doc_id	bigint	FK→knowledge_doc, NN	所属文档
content	text	NN	切片文本
embedding	vector(1024)	NN	切片向量（pgvector）

4.8 action（动作库表）

字段	类型	约束	说明
id	bigint	PK	主键

name	varchar(100)	NN	动作名称
muscle_group	varchar(50)		目标肌群
contraindication	text		禁忌与替代动作

4.9 tool_log（工具调用日志表）

字段	类型	约束	说明
id	bigint	PK	主键
message_id	bigint	FK→chat_message, NN	触发消息
tool_name	varchar(50)	NN	工具名（calc_tdee 等）
args	jsonb		入参
result	jsonb		出参

4.10 agent_config（Agent 配置表）

字段	类型	约束	说明
id	bigint	PK	主键（单例）
system_prompt	text	NN	系统提示词
model	varchar(50)	NN	模型标识（可替换）
guardrails	text		安全护栏配置（JSON）

4.11 完整性约束与规范化

- **实体完整性**：所有表均设自增主键 `id`；
- **参照完整性**：外键关联 user、exercise_plan、plan_item、chat_session、knowledge_doc 等，删除采用 RESTRICT 或级联策略（如删用户级联删其处方、会话）；
- **用户自定义完整性**：role 取值约束、plan.status 状态枚举、workout_log 数值非负检查；
- **规范化**：实体关系达到 3NF；exercise_plan.fitt_vp 与 tool_log 的入出参采用 JSONB，属可控的反规范化，换取处方结构与工具日志的灵活性。

5 对象关系映射（ORM）

实体类与库表的映射关系如下（后端采用 ORM 框架，实体类字段采用驼峰命名，库表采用下划线命名，由 ORM 自动映射）：

实体类	库表	映射说明
User	user	id↔id, username, passwordHash↔password_hash, role, nickname, createdAt↔created_at
HealthProfile	health_profile	userId↔user_id（1:1）, age, gender, heightCm↔height_cm, weightKg↔weight_kg, goal, injuryNotes↔injury_notes, parqResult↔parq_result
ExercisePlan	exercise_plan	userId↔user_id, fittVpJson↔fitt_vp（JSONB）, status, generatedAt↔generated_at
PlanItem	plan_item	planId↔plan_id, type, detail, intensity
WorkoutLog	workout_log	userId↔user_id, planItemId↔plan_item_id, weightKg↔weight_kg, note, logAt↔log_at
ChatSession / ChatMessage	chat_session / chat_message	sessionId↔session_id, role, content, citations
KnowledgeDoc / KnowledgeChunk	knowledge_doc / knowledge_chunk	docId↔doc_id, content, embedding↔vector
Action / ToolLog / AgentConfig	action / tool_log / agent_config	messageId↔message_id, toolName↔tool_name, args/result↔jsonb

5.1 映射示例（后端实体类）

```
@Entity
@Table(name = "exercise_plan")
public class ExercisePlan {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "fitt_vp", columnDefinition = "jsonb")
```

```
    private String fittVpJson;

    @Column(name = "status")
    private String status;

    @Column(name = "generated_at")
    private LocalDateTime generatedAt;

    @OneToMany(mappedBy = "plan")
    private List<PlanItem> items;
}
```

> 说明：以 Java JPA 注解为例；若后端采用 Python FastAPI + SQLAlchemy，则用 `__tablename__ = "exercise_plan"` 与 `Column("user_id", ForeignKey("user.id"))` 表达等价映射。字段命名规范保持一致。

6 物理结构设计

6.1 索引设计

表	索引	说明
user	UQ(username)	登录唯一性
health_profile	UQ(user_id)	1:1 快速定位
exercise_plan	idx(user_id, generated_at)	按用户+时间查历史处方
workout_log	idx(user_id, log_at)	进度曲线时间序列查询
chat_message	idx(session_id)	会话内消息查询
knowledge_chunk	IVFFlat/HNSW(embedding)	pgvector 向量近似检索

6.2 存储与分区

- 热数据（近 6 个月打卡、会话）常驻；历史数据按月分区归档；
- knowledge_chunk 向量维度 1024，采用 HNSW 索引平衡召回率与速度；
- 大文件（课件、图片）不入库，存 OSS 并记录 URL。

7 领域知识与交互记录存储设计

结合本项目领域 Agent 的特点，对三类关键数据的存储与管理方式说明如下：

7.1 领域知识（知识库）

- **结构化组织**：运动科学知识按「文档（knowledge_doc）→ 切片（knowledge_chunk）」两级存储，文档带分类（处方学/强度量化/能量代谢等）与来源；
- **向量化**：切片经嵌入模型生成 1024 维向量，存于 pgvector，供 RAG 相似度检索；
- **出处标注**：检索命中的切片信息写入 chat_message.citations，实现回答可溯源。

7.2 用户交互记录（会话）

- 智能体对话以 chat_session（会话）+ chat_message（消息）两级存储，消息含 role、content、citations，支持多轮上下文回放；
- 工具调用过程记入 tool_log（工具名、入参、出参），既支撑审计，也支撑后续的 Agent 行为分析。

7.3 任务执行结果（处方与追踪）

- 任务执行结果——运动处方——以 exercise_plan（处方头）+ plan_item（训练条目）存储，FITT-VP 结构存于 jsonb；
- 执行反馈——打卡与身体数据——存于 workout_log，随时间累积形成进度曲线，作为动态调整的依据。

8 数据安全与备份

- **加密**：密码哈希（不可逆）存储；健康敏感字段传输加密、最小化收集；
- **权限**：数据库账号分角色授权，应用侧以 JWT 鉴权 + 行级权限控制；
- **备份**：每日全量 + 增量备份，WAL 归档，保留 7 天，支持时间点恢复（PITR）；
- **审计**：tool_log 与操作日志保留，满足可溯源要求。

附录 A 建表 SQL（节选）

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE "user" (
    id            BIGSERIAL PRIMARY KEY,
    username      VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role          VARCHAR(20) NOT NULL DEFAULT 'user',
    nickname      VARCHAR(50),
    created_at    TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE health_profile (
    id           BIGSERIAL PRIMARY KEY,
    user_id      BIGINT UNIQUE REFERENCES "user"(id) ON DELETE CASCADE,
    age          INT,
    gender       VARCHAR(10),
    height_cm    NUMERIC(5,1),
    weight_kg    NUMERIC(5,1),
    goal         TEXT,
    injury_notes TEXT,
    parq_result  TEXT
);

CREATE TABLE exercise_plan (
    id           BIGSERIAL PRIMARY KEY,
    user_id      BIGINT NOT NULL REFERENCES "user"(id) ON DELETE CASCADE,
    fitt_vp      JSONB NOT NULL,
    status       VARCHAR(20) NOT NULL DEFAULT 'active',
    generated_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE plan_item (
    id        BIGSERIAL PRIMARY KEY,
    plan_id   BIGINT NOT NULL REFERENCES exercise_plan(id) ON DELETE CASCADE,
    type      VARCHAR(20) NOT NULL,
    detail    TEXT,
    intensity VARCHAR(50)
);

CREATE TABLE workout_log (
    id           BIGSERIAL PRIMARY KEY,
    user_id      BIGINT NOT NULL REFERENCES "user"(id) ON DELETE CASCADE,
    plan_item_id BIGINT REFERENCES plan_item(id) ON DELETE SET NULL,
    weight_kg    NUMERIC(5,1),
    note         TEXT,
    log_at       TIMESTAMP NOT NULL DEFAULT now()
);
```

```
CREATE TABLE knowledge_doc (
    id       BIGSERIAL PRIMARY KEY,
    title    VARCHAR(255) NOT NULL,
    source   VARCHAR(255),
    category VARCHAR(50)
);

CREATE TABLE knowledge_chunk (
    id        BIGSERIAL PRIMARY KEY,
    doc_id    BIGINT NOT NULL REFERENCES knowledge_doc(id) ON DELETE CASCADE,
    content   TEXT NOT NULL,
    embedding vector(1024)
);

CREATE INDEX idx_chunk_embedding ON knowledge_chunk
    USING hnsw (embedding vector_cosine_ops);
```